

Interview cheat sheet: all lessons

Every lesson's cheat sheet in one place. ♦ = my addition beyond the video; everything else is from the lessons.

01 · What Is Inference & Inference Engineering?

[Full notes](#)

Say it in 30 seconds

Inference is running a trained model token by token for every request, so fleet-wide it costs more than training. Decode is memory-bandwidth-bound: every step streams all the weights to make one token per sequence. The core trade is latency vs throughput vs cost, and batching is lever one. The job: measure, find the bottleneck, change one thing, repeat.

Only numbers worth memorizing

- **H100**: 3.35 TB/s HBM, ♦ ~989 TFLOPS BF16. **Llama 3 8B**: 16 GB in BF16, ~128 KB KV per token (taught in 02).
- **Software matters**: batching gives ~15× cheaper tokens; vLLM gets up to 24× naive HF throughput.

Derive, don't memorize

Let P = parameters, B = batch size, BW = memory bandwidth.

1. Decode speed = bandwidth ÷ bytes per step

Every decode step streams all the weights once, so time per step is bytes moved over bytes per second.

$$t_{\text{step}} \approx \frac{2P \text{ bytes}}{BW} = \frac{16 \text{ GB}}{3.35 \text{ TB/s}} \approx 4.8 \text{ ms} \Rightarrow \leq 200 \text{ tok/s per user}$$

INTUITION

Halve the bytes (8-bit weights) and the ceiling doubles. That's why precision is a speed lever.

2. Memory-bound or compute-bound? Compare FLOPs per byte to the ridge ♦

Each weight does a multiply and an add (2 FLOPs) per token and costs 2 bytes to read. One read is shared by all B sequences:

$$\frac{\text{FLOPs}}{\text{byte}} \approx \frac{2P \cdot B}{2P} = B \quad \text{vs} \quad \text{ridge} = \frac{989 \text{ TFLOPS}}{3.35 \text{ TB/s}} \approx 300$$

Batch 1 gives ~1, far below 300, so decode is memory-bound and batching is nearly free speed. Prefill puts ~500 prompt tokens through one read, above the ridge, so it's compute-bound:

$$t_{\text{prefill}} \approx \frac{N_{\text{prompt}} \cdot 2P}{\text{peak FLOPS}} = \frac{500 \times 16 \text{ GFLOP}}{989 \text{ TFLOPS}} \approx 8 \text{ ms}$$

3. Cost = what you pay per hour ÷ what you make per hour

$$\frac{\$}{1\text{M tokens}} = \frac{\$ \text{ per GPU hour}}{\text{tok/s} \times 3600} \times 10^6 \quad \frac{3}{200 \times 3600} \times 10^6 \approx 4.17 \quad \frac{3}{3000 \times 3600} \times 10^6 \approx 0.28$$

4. Why batching isn't free: every sequence brings its own KV cache ♦

A step reads the weights once plus every sequence's cache (context length C ; KV/token ≈ 128 KB, derived in 05), so steps slow down as the batch grows:

$$t_{\text{step}}(B) \approx \frac{2P + B \cdot C \cdot \text{KV/token}}{\text{BW}}$$

Prefill vs decode

	Prefill	Decode
Work	Whole prompt in one parallel pass	One token per sequence per step, serial
Bottleneck	Compute (FLOPs)	Memory bandwidth (bytes)
User sees it as	TTFT	Gap between tokens (ITL / TPOT)
Grows with	Prompt length	Output length, and batch × context (KV reads)

Symptom → lever (for "how would you fix it?" questions)

Symptom	Likely cause	First levers
High \$/token, GPU underused	Batch too small	Scheduling: batch more requests per step
Batch capped by out-of-memory	KV cache fills memory	Memory: pack KV tightly, reuse shared prefixes
Slow tokens even at low load	Too many weight bytes per step	Precision: 8/4-bit weights
High TTFT	Long prompts or queueing	Shorter prompts, more replicas (routing fixes: see 02)
Model doesn't fit one GPU	Size	Parallelism across GPUs, or precision

Rapid-fire Q&A

Question	Crisp answer
Why does batching help, and what does it cost?	One weight read serves all B sequences. The cost is KV memory and slower steps (derivation 4).
♦ Max tok/s for 70B BF16 on 8×H100?	Derivation 1: $140 \text{ GB} \div (8 \times 3.35 \text{ TB/s}) \approx 5.2 \text{ ms/step}$, so $\leq \sim 190 \text{ tok/s}$ before communication overhead.
Cost too high: where do you start?	Measure (utilization, batch size, TTFT/ITL) before changing anything; then check off-GPU wins such as a smaller model, shorter prompts, caching.

TRAP

- "Just call `generate()`" or "buy a faster GPU": both ignore the scheduler, memory and setup.
- Trading away quality: a faster but worse answer is not a win.

02 · The Inference Stack: Model to GPU to Production

Full notes

Say it in 30 seconds

A request crosses about eight layers. The model defines the math; kernels, CUDA and the GPU set how fast it runs, bounded by bytes ÷ bandwidth. The inference engine batches requests every step and manages KV cache memory. The API server, router and autoscaler pick the replica and the replica count. Nearly all time is GPU time; the upper layers decide how well it's used.

Only numbers worth memorizing

- **H100**: 80 GB HBM, 132 SMs. **Llama 3.1 8B**: 32 layers. Routing ~1 ms; decode ~5 ms/token.

Derive, don't memorize

Every derivation here is the same idea: **time = bytes ÷ bandwidth**, or **capacity = bytes ÷ bytes-per-item**. Per-step time: see 01, derivation 1. KV per token: see 05, derivation 2.

1. KV capacity = memory left after the weights ÷ cache per token ♦

$$N_{\text{tokens}} \approx \frac{\text{HBM} - \text{weights}}{\text{KV/token}} = \frac{80 - 16 \text{ GB}}{128 \text{ KB}} \approx 488\text{k} \approx 119 \text{ requests} \times 4\text{k}$$

INTUITION

That token budget, not compute, caps the batch. It's why engines manage KV in blocks and why quantizing weights or KV frees room for more users.

2. Launch overhead = kernels × launch cost, against the step time ♦

$$\frac{N_{\text{kernels}} \times t_{\text{launch}}}{t_{\text{step}}} = \frac{300 \times 4 \mu\text{s}}{4.8 \text{ ms}} \approx 25\%$$

INTUITION

Illustrative numbers, but the shape is real: at small batch, CPU launch time is a big slice of each step. Fusion (and ♦ CUDA graphs) cut it.

3. Cold start = weight bytes ÷ load bandwidth ♦

$$t_{\text{cold}} \approx \frac{16 \text{ GB}}{2 \text{ GB/s}} = 8 \text{ s}$$

INTUITION

Assumes ~2 GB/s from local disk; from network storage it's often slower. Scaling reacts late, so autoscalers must scale before the queue explodes.

Layer → question → what to watch

Layer	Answers	Tools	Watch / lever
Model + framework	What to compute	Weights, recipe, tokenizer, PyTorch	Model size, precision
Kernels + CUDA + GPU	How fast the math runs	cuBLAS, FlashAttention, CUDA	Bytes moved, fusion, launches
Inference engine	Who runs each step; where memory goes	vLLM, SGLang, TensorRT-LLM	Batch size, KV usage
API server	Speaks HTTP, tokenizes, streams	OpenAI-compatible server	Streaming latency
Router + autoscaler	Which replica; how many	Prefix-aware router	Queue length, cache hits

Symptom → layer to investigate

Symptom	Layer	First fix
GPU idle between kernels, CPU busy	Kernels	Fuse kernels; cut launches
Same math slower than expected	Kernels	Better kernels (cuBLAS, FlashAttention)
Repeated system prompts inflate TTFT	Router	Prefix-aware routing
Queue grows while CPU looks idle	Autoscaler	Scale on queue length and KV use
New replicas arrive too late	Fleet	Derivation 3: scale earlier, load faster

Rapid-fire Q&A

Question	Crisp answer
What does an inference engine add over PyTorch?	A per-step scheduler (batching) and a KV cache memory manager.
Why not autoscale on CPU?	The work is on the GPU; the CPU idles. Watch queue length and KV use.
Why is a round-robin load balancer wasteful?	It scatters shared prompts, so each replica re-prefills them.
What's a replica?	One engine on its GPU(s), behind an API server.
Where does a request's time go?	Mostly GPU (prefill, then ~5 ms per decode step); upper layers decide utilization.

TRAP

- Treating LLM serving like a stateless web app: round-robin routing and CPU-based autoscaling.
- Assuming the model file determines speed: it only defines the math.

03 · Levels of Inference Engineering

[Full notes](#)

Say it in 30 seconds

Inference work sits on four levels: the user picks and calls models, the operator makes a model fit and stay up, the optimizer gets more tokens per GPU within latency targets, and the engine builder changes the engine itself. Each level is easier if you understand the one below. The key skill is finding which level a problem lives on; cheap fixes on lower levels often beat kernel work.

Only numbers worth memorizing

- **DeepSeek-R1 (Feb 2025):** \$0.55/M input, \$2.19/M output ($\sim 4\times$), \$0.14/M cached input.
- **Batching measured:** MPT-7B/A100 0.9 $\rightarrow$ 12.5 req/s ($\sim 14\times$).

Derive, don't memorize

1. API bill = tokens $\times$ price, per token type

Each token type has its own price, and a cached prefix bills at the cached rate:

$$\text{cost} = N_{\text{cached}} \cdot p_{\text{cached}} + N_{\text{in}} \cdot p_{\text{in}} + N_{\text{out}} \cdot p_{\text{out}} \quad \Rightarrow \quad \$2.42 \rightarrow \$1.19 \text{ per 1k requests}$$

INTUITION

Put stable text first (caches match prefixes $\blacklozenge$, so any change early in the prompt invalidates everything after it), and cap output length: output tokens cost $\sim 4\times$.

The operator's fit check (params $\times$ bytes per param vs GPU memory): see 04, derivation 1.

The four levels

Level	Question	Controls	Example win
1 · User	Which model, called how?	Model, prompt, output length, streaming, caching	Stable prefix first: bill $\div 2$
2 · Operator	Does it fit and stay up?	GPUs, precision, startup, failover, rollouts	Day/night fleet sizing
3 · Optimizer	More per GPU within latency?	Batching, quantization, prefix caching, speculative decoding, PD split	$\sim 14\times$ from batching
4 · Engine builder	What should the engine do differently?	Schedulers, kernels, memory layout	Orca, FlashAttention, PagedAttention

Symptom $\rightarrow$ which level

Symptom	Level	First fix
API bill high, same instructions every request	1	Identical block at the front for cache hits
Model won't load on one GPU	2	Split across GPUs or quantize
Fits at launch, out of memory at peak	2	Budget KV for peak concurrency
Attention kernel dominates the profile	4	Kernel work (after levels 1-3 are exhausted)

Rapid-fire Q&A

Question	Crisp answer
Why do output tokens cost $\sim 4\times$ input?	Decode is serial and memory-bound; prefill is parallel (see 01, derivation 2).
Why split prefill and decode onto separate machines?	◆ Prefill is compute-bound, decode memory-bound; each group is tuned for its own job.
What did Orca, FlashAttention, PagedAttention contribute?	Per-step scheduling (continuous batching); no big attention matrix in HBM; paged KV cache (vLLM).
Is the engine builder the "senior" level?	No: levels aren't seniority, and most savings come from levels 1-3.

TRAP

- Jumping to level 4 (kernels) before checking model choice, prompts and caching.
- Sizing memory for launch traffic instead of peak.

04 · Math for Memory, Throughput & Speedups

Full notes

Say it in 30 seconds

Every number is an amount, a rate or a ratio, and $\text{time} = \text{amount} \div \text{rate}$. Weight memory is $\text{params} \times \text{bytes per param}$, so precision is the first lever for fit. Divide bytes by bandwidth and FLOPs by FLOPS to get floors; real runs land above them. Latency and throughput are reciprocals only for one stream. Speedups are $\text{before} \div \text{after}$, and Amdahl's law caps them by the fraction of time touched.

Only numbers worth memorizing

- **Bytes per param:** FP32 4, BF16 2, FP8 1, INT4 0.5.

Byte units: decimal vs binary

Decimal (SI)	Bytes	Binary (IEC)	Bytes	Binary ÷ decimal
1 KB (kilobyte)	$10^3 = 1,000$	1 KiB (kibibyte)	$2^{10} = 1,024$	1.024
1 MB (megabyte)	10^6	1 MiB (mebibyte)	$2^{20} = 1,048,576 \approx 1.049 \times 10^6$	1.049
1 GB (gigabyte)	10^9	1 GiB (gibibyte)	$2^{30} = 1,073,741,824 \approx 1.074 \times 10^9$	1.074
1 TB (terabyte)	10^{12}	1 TiB (tebibyte)	$2^{40} = 1,099,511,627,776 \approx 1.100 \times 10^{12}$	1.100

The "i" means binary: kibi = **kilo binary**, mebi = **mega binary**, and so on. Say "kibibyte" (KIB-ee-byte), "gibibyte" (GIB-ee-byte).

- **GB → GiB:** divide by 1.074. 16 GB = **14.9 GiB**; an 80 GB H100 = **74.5 GiB**.
- ♦ Datasheet rates are decimal: **1 TB/s = 1 GB/ms = 1 MB/μs**, so 3.35 TB/s moves 16 GB in $16 \div 3.35 \approx 4.8$ ms.

Derive, don't memorize

1. Weights = params × bytes per param; check fit first

$$\text{weights} = N \times \frac{\text{bits}}{8} : \quad 70\text{B} \rightarrow 140 \text{ (BF16)}, \quad 70 \text{ (FP8)}, \quad 35 \text{ (INT4)} \text{ GB vs } 80 \text{ GB}$$

INTUITION

"Fits" means weights + KV cache at peak + runtime (KV capacity: see 02, derivation 1). Too big: split across GPUs or quantize.

2. Time = amount ÷ rate, and the slower floor wins ♦

Each token must both move bytes and do math; whichever takes longer sets the floor:

$$t \geq \max \left(\frac{\text{bytes}}{\text{BW}}, \frac{\text{FLOPs}}{\text{FLOPS}} \right) \quad 70\text{B FP8} : \frac{70}{3350} \approx 21 \text{ ms} \quad \text{INT4} : \approx 10 \text{ ms}$$

INTUITION

A measurement below the floor means an assumption broke (fewer bytes moved, or a bad measurement). Why bytes usually win: see 01, derivation 2.

3. Speedup = before ÷ after; Amdahl caps it

$$S = \frac{t_{\text{before}}}{t_{\text{after}}} \quad S_{\text{overall}} = \frac{1}{(1-p) + p/s} \leq \frac{1}{1-p}$$

INTUITION

"x% faster" = $1 + x$; "x% less time" = $1 / (1 - x)$. 30% of a step made 3× faster gives 1.25×; infinitely faster gives 1.43×.

Symptom → diagnosis

Symptom	Diagnosis
Tool shows 14.9 for a 16 GB file	GiB vs GB, not a bug
Measured faster than the bandwidth floor	Fewer bytes moved than assumed, or a broken measurement
Kernel 3× faster, end to end barely moves	Small p : Amdahl
"1,800 tok/s" but users say it's slow	Throughput across many streams; ask per-stream speed

Rapid-fire Q&A

Question	Crisp answer
FLOPs vs FLOPS?	Count (what a model costs) vs per second (what a GPU supplies).
Why are napkin numbers floors?	Datasheet peaks are never sustained, so real runs are slower.
20 ms → 15 ms: how much faster?	1.33× (25% less time), not 1.25×.
Is a throughput gain a latency gain?	Only for one stream; with batching they decouple.

TRAP

- Adding speedups (" $2\times + 2\times = 4\times$ "): compose by time accounting instead.
- Quoting a speed floor for a model that doesn't fit.

05 · Dot Products, Softmax & Attention Math

Full notes

Say it in 30 seconds

Attention scores are dot products of the new token's query with every past key, scaled by $\sqrt{d}$ and softmaxed into weights that mix the values. Per decode token that's little arithmetic but a full read of every stored key and value, about 4 ops per byte, so decode attention is memory-bound. K/V grows 128 KB per token per user in Llama 3 8B, overtaking the weights at long context or big batch. Prefill instead faces an n^2 score matrix, which FlashAttention avoids with an online softmax.

Only numbers worth memorizing

- **Llama 3 8B attention:** 32 layers, 32 query heads, 8 KV heads (GQA), head dim 128. **FP16 max:** 65,504.

Derive, don't memorize

Let L = layers, H = heads, d = head dim, n = context length.

1. Attention ops per decode token = dots $\times$ cost per dot $\times$ 2 (scores + weighted sum)

$$\text{ops} \approx L \cdot H_q \cdot n \cdot 2d \cdot 2 = 32 \cdot 32 \cdot 4096 \cdot 256 \cdot 2 \approx 2.1 \text{ B} \quad (\text{vs } \approx 15 \text{ B for weights})$$

2. KV bytes per token: one K and one V per KV head per layer

$$\text{KV/token} = 2 \cdot L \cdot H_{\text{kv}} \cdot d \cdot \text{bytes} = 2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 128 \text{ KB} \quad \Rightarrow \quad \frac{2.1 \text{ B ops}}{4096 \times 128 \text{ KB}} \approx 4 \text{ ops/byte}$$

INTUITION

4 $\ll$ ~ 300 ridge (01, derivation 2): memory-bound. K/V matches the 16 GB weights at $16 \text{ GB} \div 128 \text{ KB} \approx 122\text{k}$ tokens for one user, or $\sim 4\text{k}$ each for 32 users. GQA's 8 KV heads instead of 32 cut this $4\times$ $\blacklozenge$.

3. Prefill scores = n^2 per head

$$n^2 \cdot H \cdot 2 \text{ B} = 8192^2 \times 32 \times 2 \approx 4.3 \text{ GB per layer}$$

INTUITION

Quadratic scratch is why FlashAttention tiles keys and never writes the square to memory.

4. Online softmax: running max m and sum l , rescaled when m rises

$$m' = \max(m, m_{\text{blk}}) \quad l' = l \cdot e^{m - m'} + \sum_{\text{blk}} e^{x - m'}$$

INTUITION

Blocks [2, 1] then [3, 0]: $l = 1.37 \rightarrow 1.37 \cdot e^{-1} + e^0 + e^{-3} \approx 1.55$, identical to the full row.

Decode attention vs prefill attention

	Decode (1 new token)	Prefill (n prompt tokens)
Queries	1 per head	n per head
Problem	Reading all K/V (bytes)	$n \times n$ score matrix (scratch)
Grows with	Context $\times$ batch	Prompt length squared
Main fixes	GQA, KV quantization $\blacklozenge$, paged KV	FlashAttention (tiling + online softmax)

Rapid-fire Q&A

Question	Crisp answer
Why divide by $\sqrt{d}$?	Random q-k has variance d; unscaled scores make softmax one-hot.
Why subtract the max in softmax?	FP16 overflows ($e^{12} > 65,504$); the weights are unchanged.
What does GQA save?	K/V memory and bandwidth: 8 shared KV heads instead of 32.
Is attention the bottleneck?	Only at long context or big batch; otherwise the weights dominate.

TRAP

- Counting attention FLOPs instead of K/V bytes for decode.
- Calling softmax "cheap": its whole-row dependency dictates kernel design.